

Sortowanie przez scalanie

„[] opanowanie teorii jest zaledwie niezbędnym wstępem”

Grzegorz Szymkowiak

Czym jest sortowanie
przez scalanie?

Sortowanie przez scalanie

- Sortowanie przez scalanie (ang. merge sort) to algorytm działający poprzez rekurencyjne dzielenie tablicy na mniejsze części, a następnie scalanie ich w posortowaną całość

38	27	43	3	9	82	10
----	----	----	---	---	----	----

Algorytm

Algorytm

1. Jeśli tablica zawiera jeden element lub jest pusta, uznaje się ją za posortowaną.
2. W przeciwnym razie tablica jest dzielona na dwie możliwie równe części (z dokładnością do jeden).
3. Dla każdej części ponownie wykonywany jest algorytm Merge Sort, czyli następuje dalszy podział na mniejsze fragmenty.
4. Gdy wszystkie fragmenty zostaną rozbite do pojedynczych elementów, rozpoczyna się proces scalania.

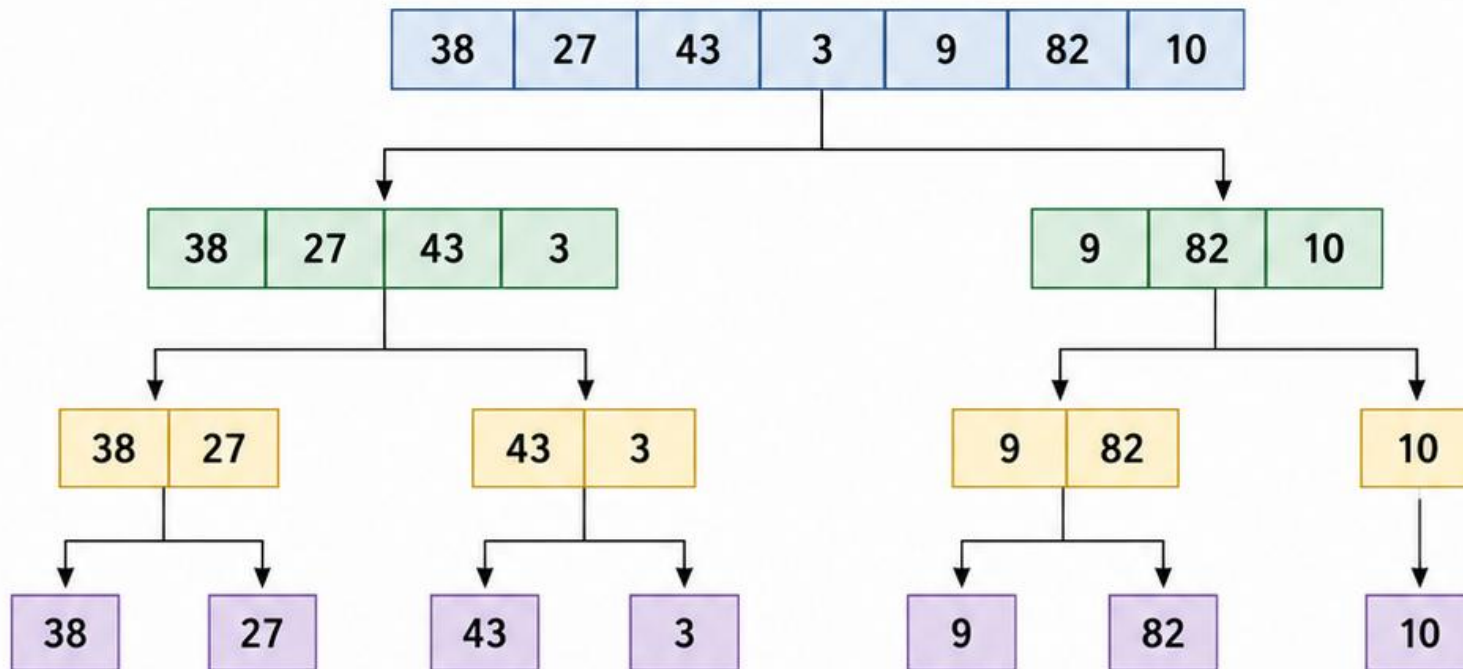
Algorytm

4. Podczas scalania porównywane są kolejne elementy dwóch posortowanych części. Do nowej tablicy zawsze przenoszony jest mniejszy z porównywanych elementów.
5. Po wyczerpaniu jednej części do wyniku dopisywane są wszystkie pozostałe elementy drugiej części.
6. Proces scalania jest powtarzany aż do połączenia wszystkich fragmentów w jedną posortowaną tablicę.

Algorytm

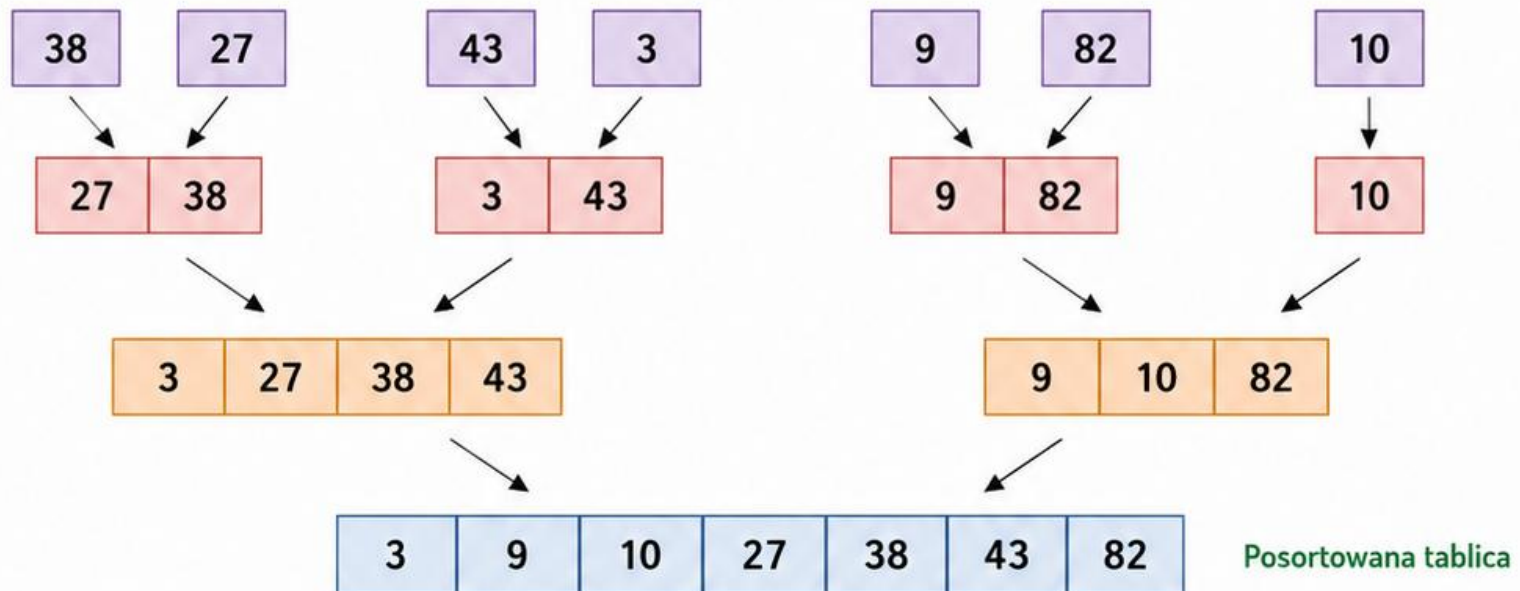
1. Jeśli tablica zawiera jeden element lub jest pusta, uznaje się ją za posortowaną.
2. W przeciwnym razie tablica jest dzielona na dwie możliwie równe części (z dokładnością do jeden).
3. Dla każdej części ponownie wykonywany jest algorytm Merge Sort, czyli następuje dalszy podział na mniejsze fragmenty.
4. Gdy wszystkie fragmenty zostaną rozbite do pojedynczych elementów, rozpoczyna się proces scalania.

Algorytm



Podział – dzielenie tablicy na coraz mniejsze części

Algorytm



Scalanie – łączenie posortowanych części (kluczowy działanie algorytmu)

Algorytm

1. Wyznacz środek
2. Sortuj lewą połowę
3. Sortuj prawą połowę
4. Scal obie posortowane połowy

Pseudokod

Pseudokod

- Realizacja algorytmu wymaga dwóch funkcji:
 - `mergSort` – funkcja sortująca
 - `merge` – funkcja scalająca

Funkcja MergeSort

```
funkcja MergeSort(A[],p,k)
    jeśli p < k to
        s ← (p+k) div 2 // indeks środka fragmentu
        MergeSort(A,p,s) // sortuj lewy fragment
        MergeSort(A,s+1,k) // sortuj prawy fragment
        Merge(A,p,s,k) // scalaj posortowane fragmenty
```

- A[] – tablica
- p – indeks początku
- k – indeks końca
- s – indeks środka

Funkcja Merge

funkcja Merge ($A[]$, p, s, k)

```
i ← p
j ← s + 1
m ← p
dopóki i ≤ s oraz j ≤ k wykonuj
    jeśli A[i] < A[j] to
        B[m] ← A[i]
        i ← i + 1
    w przeciwnym przypadku
        B[m] ← A[j]
        j ← j + 1
    m ← m + 1
dopóki i ≤ s wykonuj
    B[m] ← A[i]
    i ← i + 1
    m ← m + 1
dopóki j ≤ k wykonuj
    B[m] ← A[j]
    j ← j + 1
    m ← m + 1
dla i ← p, ..., k wykonuj A[i] ← B[i]
```

Implementacja

```
static void MergeSort(int[] A, int p, int k) {  
    if (p < k) {  
        int s = (p + k) / 2;  
  
        MergeSort(A, p, s);  
        MergeSort(A, s + 1, k);  
  
        Merge(A, p, s, k);  
    }  
}
```



```

static void Merge(int[] A, int p, int s, int k) {
    int[] temp = new int[k - p + 1];
    int i = p, j = s + 1, t = 0;

    while (i <= s && j <= k) {
        if (A[i] <= A[j]) {
            temp[t] = A[i]; i++;
        } else {
            temp[t] = A[j]; j++;
        }
        t++;
    }

    while (i <= s) {
        temp[t] = A[i]; i++; t++;
    }

    while (j <= k) {
        temp[t] = A[j]; j++; t++;
    }

    for (int x = 0; x < temp.Length; x++) A[p + x] = temp[x];
}

```

Złożoność obliczeniowa

Złożoność obliczeniowa

- Merge Sort ma gwarantowaną złożoność czasową $O(n \log n)$ niezależnie od stopnia uporządkowania danych, ale wymaga dodatkowej pamięci $O(n)$ na potrzeby procesu scalania.

Złożoność obliczeniowa

Przypadek	Złożoność
Najlepszy	$O(n \log n)$
Średni	$O(n \log n)$
Najgorszy	$O(n \log n)$

www.szymkowiak.tychy.pl

KONIEC

Grzegorz Szymkowiak